BVS1 – Betriebssysteme

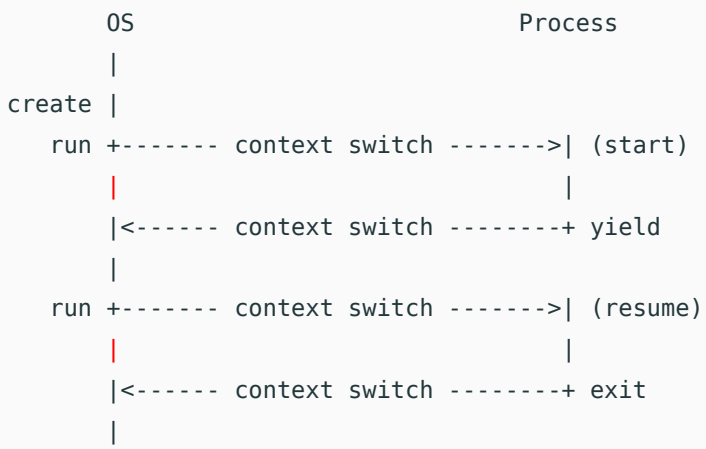
Lecture 5 – Entering the OS · Winter Semester 2026/2027

Stavros Kousidis

TH Köln · ICCT

The Mental Model of Our Current OS

The Lifetime from Lecture 3 – Revisited From Today's Perspective



Why does our current OS line continue in red while the process runs?

CC BY-SA 4.0

What If the Process Never Yields?

```
for (;;) { /* keep computing */ }
```

Our CP1 OS regains control only when the process calls `process_yield()` or `process_exit()`.

If it never calls, our OS never runs again.

Today: the first half of the answer.

CC BY-SA 4.0

From Calls to Traps

Two Halves of the Answer to a Never-Yielding Process

```
Lecture 5: the process asks       ecall   -> CPU -> OS
Lecture 6: an event interrupts    timer   -> CPU -> OS
```

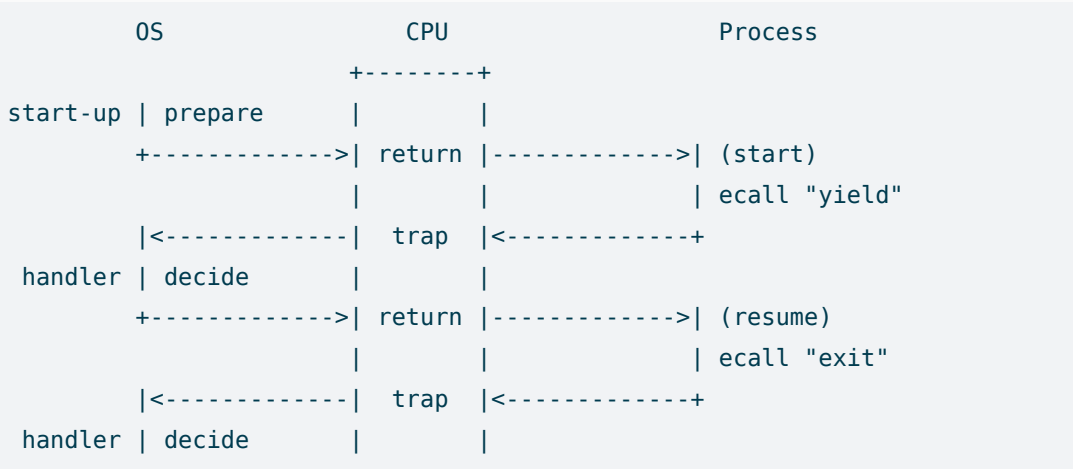
Both use **the same entry path into the OS**.

Today's question: how can the CPU transfer control to the OS?

Today's goal: refactor the yielding mechanism so that the OS really discontinues while the process runs: no more red lines.

CC BY-SA 4.0

The Lifetime Through the CPU



The CPU only traps and returns. **Where did the red lines go?**

What the CPU Provides for Traps and Returns

Kind	Provided by the CPU	Used how
state	`mtvec`, `mepc`, `mcause`, `mstatus`, `mscratch`	read/written by the OS
instructions	`csrr`, `csrw`, `csrs`, `csrrw`, `mret`	executed by the OS
	`ecall`	executed by the process
rules	on a trap: record, interrupts off, jump	automatic, on an event
	on a return: restore, interrupts on, continue	when the OS runs `mret`

The state lives in **CSRs** (control and status registers), read and written only by the `csr…` instructions. `mret`: machine return. `ecall`: environment call.

The OS configures the state, uses the instructions, and relies on the rules.

Trap and Return: What the CPU Does

Recall: Why a Function Call Is Not Enough

In CP1 the OS got control back only through a **call** the process made: `process_yield()`.

A call happens only if the running code **decides** to make it:

- the caller chooses the moment;
- the call instruction sets `ra`: where to continue;
- the calling convention says which registers may be lost.

A process in `for (;;)` never calls. An event between two instructions offers **none of this**.

We need an entry into the OS that does not depend on the process's cooperation.

A Trap: CPU-Forced Entry

A **trap** is a transfer of control to an OS-defined handler that the CPU performs itself.

	synchronous	asynchronous
caused by	the current instruction	an independent event
where	at that instruction, every run	between two instructions, varies
RISC-V name	exception	interrupt
"trap `pc`"	the causing instruction	the next instruction, not yet executed
in BVS1	`ecall`, illegal instruction (today)	timer (Lecture 6)

Recall (Lecture 4): the `pc` (program counter) is **architectural state**, not one of the general-purpose registers `x0`–`x31`; no instruction stores it to memory like a register.

CC BY-SA 4.0

Who Can Save the pc?

Suppose the CPU simply jumped into the OS, to the address the OS chose earlier:

```
process   ecall                <- must continue after this
OS        first instruction    <- live pc is here now
```

The OS code can only compute addresses near **its own** position.

It can no longer recover where the process was, unless the CPU captured that address at the moment of the jump.

CC BY-SA 4.0

What the CPU Does on a Trap: Record Where and Why

On a trap, the CPU first records **where** and **why**, in two **CSRs** (control and status registers):

CSR	Stands for	Holds
`mepc`	Machine Exception Program Counter	the **trap pc** (our term)
`mcause`	Machine Cause	bit 63: interrupt; bits 62..0: code

`mcause`	Meaning	`mepc` holds
2	illegal instruction	that instruction
11	`ecall` from M-mode	the `ecall`
7, interrupt bit set	timer (next week)	next instruction, not yet executed

What the CPU Does on a Trap: Interrupts Off

Two bits of the CSR `mstatus` (machine status), bits 3 and 7:

Bit	Stands for	Means
`MIE`	Machine Interrupt Enable	interrupts allowed (1) or not (0)
`MPIE`	Machine Previous Interrupt Enable	the value `MIE` had before the trap

```
MPIE <- MIE     remember whether interrupts were allowed
MIE  <- 0       no interrupt now
```

Nobody interrupts the OS while it saves the process's state.

What the CPU Does on a Trap: Jump

```
pc <- mtvec        the address held in the CSR mtvec
```

`mtvec` (Machine Trap-Vector): **one** address for **every** trap.

Who put it there? The OS, once at start-up:

```
csr_write(mtvec, (uintptr_t)trap_entry);    /* csrw */
```

`trap_entry`: a label in the provided `trap.S`. Like every symbol (`nm -n`, Lecture 4), it gets its address from the linker.

The OS decides once where all traps enter: at `trap_entry`.

What the CPU Does on a Return

On a trap	On a return (`mret`, machine return)
record where: `mepc <- trap pc`	continue there: `pc <- mepc`
record why: `mcause <- reason`	nothing: `mcause` was only for the OS
interrupts off: `MPIE <- MIE`, `MIE <- 0`	reversed: `MIE <- MPIE`

If the OS changes `mepc` before `mret`, execution continues elsewhere.

The `pc` is architectural state, never written like a register: `mret` sets it from `mepc`.

The CPU neither saves nor restores the process's registers: **the OS saves them after the trap and restores them before `mret`**.

Between Trap and Return: The OS's Mechanisms

Why `ra`, `sp`, `s0–s11` No Longer Suffice

CP1 saved 14 values: **enough at a call boundary**. Compiled code follows the RISC-V calling convention (ABI): across a call, only `sp` and `s0–s11` must survive; `ra` holds the continuation. Saving all 31 would also have been correct, just unnecessary.

A trap is **not a call**: the convention gives no guarantee there.

```
ecall in process_yield()   at a call boundary
illegal instruction        anywhere in the code
timer (Lecture 6)          between any two instructions
```

One entry path serves all traps: it must save all 31 writable integer registers.

The Trap Frame Replaces `struct context`

```
struct trap_frame {
    uintptr_t regs[32];   /* regs[i] = xi; x0 unused */
    uintptr_t pc;         /* continuation address */
};
```

A **trap frame** is the complete saved state of an execution that trapped. Its `pc` is the saved continuation address.

	CP1 `struct context`	`struct trap_frame`
registers	`ra`, `sp`, `s0`–`s11`	31 integer registers
continuation	`ra`, restored by `ret`	`pc`, via `mepc` and `mret`
field	`struct context context;`	`struct trap_frame frame;`

The Trap Path Replaces `context_switch()`: Save

```
trap_entry:                     # mscratch = &p.frame, set by OS
    csrrw t6, mscratch, t6      # swap: t6 = &p.frame
    sd    x1, 8(t6) ...         # x1 .. x30 into the frame
    csrr  t5, mscratch          # t6 of the process
    sd    t5, 248(t6)           #   as x31
    csrr  t5, mepc              # the trap pc
    sd    t5, 256(t6)           #   as frame->pc
    la    sp, _stack_top        # only now: the OS stack
    mv    a0, t6                # for trap_handler: &p.frame
    call  trap_handler          # runs on the OS stack
```

The CPU leaves `sp` unchanged. The registers go into `p.frame`; only then does `la sp` switch to the OS stack for `trap_handler()`.

CC BY-SA 4.0

The Return Path Replaces `context_switch()`: Restore

```
    call  trap_handler       # returns a0 = frame to restore
os_start:                    # os_run() enters here, too
    csrw  mscratch, a0       # a0: result of trap_handler
    ld    t5, 256(a0)
    csrw  mepc, t5           # where to continue
    ...                      # mstatus: MPP, MPIE
    ld    x1, 8(a0) ...      # x1 .. x31 from the frame, not a0
    ld    x10,  80(a0)       # a0 last: it held the frame
    mret                     # pc <- mepc, MIE <- MPIE
```

The return path is one piece of code with two ways in: after every trap, and once from `os_run()` for the first start.

`trap_handler()`: Return the Frame to Restore

```
struct trap_frame *trap_handler(struct trap_frame *trapped);
```

Called by `call trap_handler` in the trap path, with `a0` = `&p.frame`.

```
trapped     -> saved state of the process that trapped
return      -> state that runs next
```

Returned frame	Meaning
the same one	resume
another one	schedule (Lecture 7)

Refactoring `main()` and `process_yield()` to Use Traps

`ecall`: A Trap on Purpose

`ecall` is an ordinary instruction whose only effect is a trap. `process_yield()`, compiled:

```
process_yield:
    li   a0, 0
    li   a1, 0
    li   a7, 1          # SYS_YIELD
    ecall               # -> mtvec, mcause = 11
    ret
```

A **system call** is a request to the OS made with `ecall`: `a7` selects the service, `a0`/`a1` carry arguments. `process_yield()` is the C function that makes the system call `yield`.

Continue After the `ecall`

```
trap_entry:   p.frame.pc = mepc   trap pc: the ecall itself
trap_handler: p.frame.pc += 4     now the continuation address
os_start:     mepc = p.frame.pc; mret
```

An `ecall` is always 4 bytes. Interrupts get no `+4`: there `mepc` already names the instruction still to execute.

What happens if the OS forgets `pc += 4`?

`yield` and `exit` as System Calls

System call	`a7`	OS action
`yield`	1	report, `RUNNING -> READY`, resume
`exit`	0	record `a0` as status, `-> FINISHED`

```
os: process 1 yielded
os: process 1 exited 0
```

The `os:` lines now come from the trap handler.

Inside `trap_handler()`

```
uintptr_t cause = csr_read(mcause);   /* the reason */
if (cause == EXC_ECALL_FROM_M) {      /* 11: ecall */
    trapped->pc += 4;
    long a0 = trapped->regs[REG_A0];
    switch (trapped->regs[REG_A7]) {
    case SYS_YIELD: return reschedule();
    case SYS_EXIT:  return finish_current(a0);
    }
}
report_unexpected(cause, trapped->pc);
return finish_current(EXIT_STATUS_TRAP);
```

The OS does not know why it was entered until it reads `mcause`.

CC BY-SA 4.0

Unexpected Traps

A faulty instruction in the process enters through the **same** `mtvec`. Inserted into `worker()`:

```
__asm__ volatile (".word 0");   /* illegal instruction */
```

`mcause` = 2, so `trap_handler()` reaches its last two lines (previous slide):

```
trap: cause 2 at 0x80000136 in process 1
```

One faulty process does not take down the OS.

First Entry: Prepare a Frame

A process that never ran has no saved state yet.

CP1 initial context	Trap-frame equivalent
`p.context.ra = process_start`	`p.frame.pc = process_start`
`p.context.sp = stack top`	`p.frame.regs[2] = stack top`
`p.context.s[i] = 0`	all other `p.frame.regs[i] = 0`

The new process looks as if it had trapped into the OS just before its first instruction.

CC BY-SA 4.0

Three Entries

```
void main(void)                 /* the OS's start-up */
{
    process_create(&p, worker);
    os_run();                   /* never returns */
}
```

```
_start      CPU's entry      once, at reset; calls main()
main()      OS start-up      once; never returns
trap_entry  OS entry         every trap, via mtvec
```

`main()` is where the OS begins, not where it lives.

Why Must p Become static?

Exercise 2 declared `struct process p;` inside `main()`.

```
OS stack
+-------------------------------------+  _stack_top <- sp
| main()'s stack frame: p, p.stack ... |
+-------------------------------------+
```

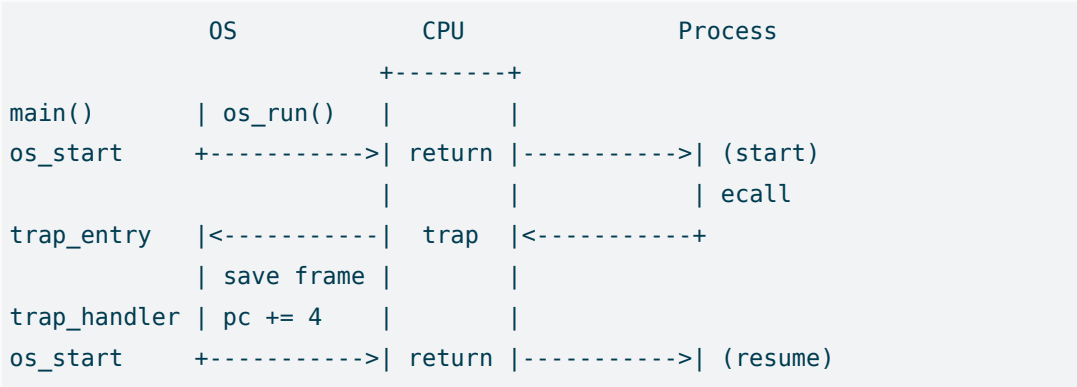
Every trap executes `la sp, _stack_top`.

Hint: `trap_handler()` then builds its stack frames downward from `_stack_top`, and `main()` is never resumed.

What happens to p at the first trap?

The Whole Path

The Lifetime in Detail



`exit` takes the same path; its handler returns the idle process's frame.

The CPU traps and returns. The OS saves, decides, and restores.

Our Trap Model: Who Does What

Step	Who	What
trap	CPU	`mepc`, `mcause`; interrupts off; `pc <- mtvec`
save	`trap_entry`	registers and trap `pc` into the frame
decide	`trap_handler()`	read `mcause`, return a frame
restore	`os_start`	registers and `mepc` from the frame
return	CPU, `mret`	`pc <- mepc`; interrupts on

Provided: `trap_entry`, `os_start` assembly.

At CP2: your own `trap_handler()`, with timer and scheduler (following Lectures 6 and 7).

Outside the model: privilege modes, nested traps, virtual memory, other interrupts.

CC BY-SA 4.0

Exercise 4: The Same Application Through a Trap

Bring `process.h` and `main.c` from Exercise 2; `worker()` and `permute()` stay **unchanged**.

Task	You do
1	understand `trap.S` with your partner: trap path, return path
2	update `process.h`: `struct trap_frame frame`, `os_run()`
3	port `main.c`: the process object, `os_run()`; predict, run
4	draw the lifetime of your run with the CPU box
5	trace one `ecall` with real addresses from `objdump`
6	diagnose three failures

Same application, same permutation: only the way into the OS has changed.

CC BY-SA 4.0

Key Ideas

- **Trap:** the CPU transfers control into the OS itself.
- **`mtvec`, `mepc`, `mcause`:** where to go, the trap `pc`, the reason.
- **Trap frame:** 31 registers plus the continuation address.
- **Trap path, return path:** replace `context_switch()`.
- **System call:** a request made with `ecall`; the handler adds 4.
- **`trap_handler()`:** reads `mcause`, returns the next frame.
- **`main()`:** start-up only; then the OS is entered at `trap_entry`.

Next: the timer enters the OS **without** the process asking.

CC BY-SA 4.0

References

Selected references:

- RISC-V International: *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture* — machine-mode CSRs, traps, `mret`.
- RISC-V International: ABIs Specification — caller- and callee-saved registers.
- Arpaci-Dusseau and Arpaci-Dusseau: *Operating Systems: Three Easy Pieces*, chapter *Mechanism: Limited Direct Execution*.

CC BY-SA 4.0